

Self-Paced Curriculum Reinforcement Learning for Autonomous Superbike Racing in Simulation

Luca Ghisi
Università degli Studi di Milano
Milan, Italy

Jacopo Essenziale
Funny Tales
Milan, Italy

Carlo D'Eramo
University of Würzburg
Würzburg, Germany

Matteo Luperto
Università degli Studi di Milano
Milan, Italy
matteo.luperto@unimi.it

Abstract—Autonomous Racing has seen remarkable progress through deep Reinforcement Learning (RL), primarily for four-wheeled vehicles. However, motorbikes introduce substantially greater complexity due to the need to manage balance and lean angle, in addition to more reactive steering and throttle control, and a smaller weight. In this work, we present a framework for training an autonomous agent to race a superbike in VRider SBK, a physics-accurate Unity-based motorbike simulator. Our approach integrates Soft Actor-Critic (SAC) with Self-Paced curriculum Deep reinforcement Learning (SPDL), which dynamically generates progressively more challenging tasks based on the agent’s performance — without requiring manual curriculum design. The agent’s state space comprises proprioceptive features extended with lean-angle history, along with global track features via course points. The reward signal is shaped to encourage progress along the track while penalizing instability-inducing behaviors specific to two-wheeled dynamics. Preliminary experimental results demonstrate that SPDL outperforms SAC alone in training efficiency, lap time, and driving stability across multiple tracks and motorbike models, establishing a first baseline for RL-based autonomous motorbike racing.

I. INTRODUCTION

Autonomous Racing is a demanding subfield of autonomous driving, requiring an agent to perceive its environment, plan an optimal trajectory, and execute precise vehicle control at the physical limits of the vehicle. In recent years, end-to-end deep Reinforcement Learning (RL) has emerged as a compelling alternative to classical modular pipelines — composed of perception, planning, and control modules — showing super-human performance in high-fidelity simulators such as Gran Turismo Sport and Gran Turismo Sport 7 [1]–[4].

Despite significant progress for four-wheeled vehicles, motorbike racing remains largely unexplored in the RL literature. A racing motorbike introduces fundamentally different physical and dynamical challenges compared to a race car: besides steering and throttle/brake control, the agent must actively manage the *lean angle* (banking angle), which is essential to counteract centrifugal forces when cornering, as can be seen in Fig. 1. Failure to control the lean angle leads to loss of balance and falling — a failure mode absent in four-wheeled vehicles. At the same time, racing bikes are subject to stronger dynamics in terms of breaking and acceleration than most racing car equivalents, due to a higher power to weight ratio.

In this paper, we present a framework for training an autonomous RL agent to race a superbike in VRider SBK¹, a



Fig. 1. An example of the view from VRider SBK game (top) and a rendering of the game from a third-person point of view (bottom), showing the motorbike while banking to go through a curve at high speed.

realistic motorbike simulator modelling real-world Superbike World Championship tracks and physics. Our contributions are:

- A Reinforcement Learning architecture tailored to two-wheeled vehicles, including a rich state space with lean-angle history and a reward signal designed to handle motorbike-specific instabilities.
- Integration of Self-Paced Curriculum Learning (SPDL) [5]–[7] with Unity ML-Agents, enabling dynamic and automatic curriculum generation within a game simulation.
- Preliminary experimental evaluation demonstrating that SPDL achieves faster convergence, lower lap times, and more stable driving compared to SAC alone, with results generalized across multiple tracks and motorbike models.

¹<https://vridergame.com/>

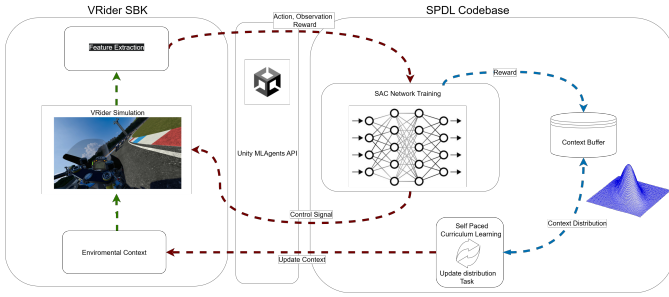


Fig. 2. The architecture of our system.

II. RELATED WORK

A. Reinforcement Learning for Autonomous Racing

The application of RL to autonomous racing has advanced rapidly. Fuchs *et al.* [1] achieved super-human lap times in Gran Turismo Sport using SAC, introducing a dense progress-based reward and kinetic-energy wall-contact penalty. Song *et al.* [8] extended this to overtaking via a manually designed three-stage curriculum. Wurman *et al.* [2] introduced GT So-phy, leveraging QR-SAC with complementary training across diverse environment configurations to produce a champion-level racing agent. More recently, Vasco *et al.* [3] and Lee *et al.* [4] incorporated vision-based inputs with asymmetric actor-critic architectures. Generalization across vehicles and conditions was addressed by Grooten *et al.* [9] via the SPARC algorithm. All existing work, however, focuses exclusively on four-wheeled vehicles.

B. Self-Paced Curriculum Learning

Self-Paced Deep reinforcement Learning (SPDL) [5]–[7] extends contextual RL by automatically generating task distributions of increasing difficulty based on the agent’s current performance, rather than relying on manually designed task sequences [8]. The algorithm maintains a context distribution $p(c|\nu)$ over a contextual MDP and optimizes:

$$\max_{\nu} \mathbb{E}_{p(c|\nu)} [J(\pi, c)] - \alpha D_{\text{KL}}(p(c|\nu) \parallel \mu(c)) \quad (1)$$

where $\mu(c)$ is the target task distribution and α controls the trade-off between reward maximization and convergence to the target. A KL divergence constraint prevents abrupt jumps between consecutive distributions, ensuring stable curriculum progression.

III. PROBLEM FORMULATION AND SIMULATION

VRider SBK is a Unity-based superbike simulator developed by Funny Tales, featuring 14 real-world Superbike championship tracks and five bike models (e.g., Ducati Panigale V4 R, Kawasaki Ninja ZX-10RR). The game is designed for VR, thus its physics is accurately modelled to reduce uncanny experience and sickness by users. The simulator provides rich data through providers for vehicle state (position, velocity, rotation), track data (median and ideal splines, track limits, checkpoints), collision and fall events, and lap timing. The

integration with RL is realised through Unity ML-Agents [10], combined with a Gym wrapper [11] and Stable Baselines 3 [12]. An overview of the system is provided in Fig. 2.

A. State Space

The agent’s observation vector combines proprioceptive and global features, all normalized to $[-1, 1]$.

Proprioceptive features: linear velocity (with longitudinal component), angular velocity, current and previous steering and lean (banking) angles, lean-angle θ , tyre slip ratios, previous control actions, and orientation relative to the followed trajectory.

Global features: 60 equally-spaced 3D course points sampled along the current trajectory (center line and left/right track limits), computed via Catmull-Rom spline interpolation at the agent’s current speed. This gives the agent a dynamic, speed-adapted look-ahead of the track shape.

The inclusion of lean-angle history and its deltas is motivated by the dynamics of motorbike control: the agent must anticipate and suppress oscillatory lean behavior that can lead to loss of control.

B. Action Space

Following the autonomous racing literature [1], two continuous actions in $[-1, 1]$ are used: (i) steering, which also drives the lean angle of the bike, and (ii) a combined throttle/brake signal, encoding the constraint that both are not applied simultaneously.

IV. REWARD DESIGN

The reward signal must be dense and guide the agent to minimize lap time while managing the instability-prone dynamics of a two-wheeled vehicle. The total reward at each timestep is:

$$\begin{aligned} r = & \lambda_{\text{way}} \lambda_{\text{prog}} * r_{\text{prog}} + \lambda_o * r_{\text{off}} + \lambda_w * r_{\text{wall}} + \lambda_s * r_{\text{steer}} + \\ & \lambda_b * r_{\text{bank}} + \lambda_{av} * r_{\text{AngVel}} + \lambda_{h_s} * r_{h_s} + \\ & \lambda_{h_b} * r_{h_b} + \lambda_{\text{maxb}} * r_{\text{maxb}} + \lambda_{\text{back}} * r_{\text{backwards}} + \\ & \lambda_{\text{still}} * r_{\text{still}} + \lambda_{\text{slip}} * r_{\text{slip}} \end{aligned} \quad (2)$$

Course progress $r_{\text{prog}} = \text{cp}_t - \text{cp}_{t-1}$: the displacement along the followed trajectory between consecutive timesteps ($\Delta t = 0.2$ s), following [1]. A wrong-way multiplier $\lambda_{\text{way}} = -1$ negates this reward when the bike’s heading deviates more than 90° from the track tangent (and is equal to 1 otherwise).

Off-track penalty $r_{\text{off}} = -(o_t - o_{t-1})|v_t|$: penalizes time spent with a tyre outside the track limits, weighted by speed.

Wall penalty $r_{\text{wall}} = -(w_t - w_{t-1})|v_t|$: kinetic-energy-based wall contact penalty [1].

Stability penalties: Steering (r_{steer}), banking (r_{bank}), and angular velocity AngVel penalties each penalize large angle changes between consecutive steps ($-|\theta_t - \theta_{t-1}|$). A history h penalty based on a sigmoid function of accumulated angle deltas discourages oscillatory behavior over recent steps and is used for steering (h_s) and banking (h_b). A max-banking maxb penalty prevents extreme lean exploitation. Tyre slip ,

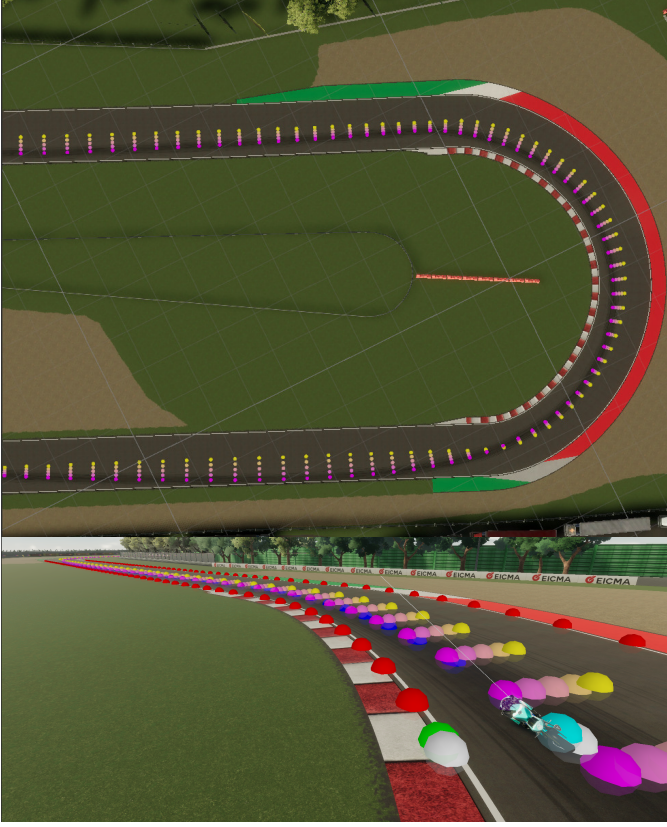


Fig. 3. (top) The trajectories followed by the motorbike, from the *median* track (yellow) to the ideal one (purple). (bottom) A close-up of the motorbike following the trajectories.

standstill, and *backward* motion penalties complete the set. Episode failure (reset) is triggered if any penalty timer expires or if the bike falls.

V. SELF-PACED CURRICULUM AND CONTEXT VARIABLES

The key insight of our curriculum is that the trajectory the agent follows during training can serve as the difficulty axis: starting from the track *median* (centerline) and progressing toward the *ideal* racing line (optimal in-out-in trajectory for minimum lap time).

Three context variables define the current task:

- c_1 : Distance from the median trajectory (higher = harder).
- c_2 : Distance from the ideal trajectory (lower = harder).
- c_3 : Maximum speed limit.

The context triple (c_1, c_2, c_3) determines which interpolated spline the agent should follow via:

$$\alpha = \frac{1}{2} \left(\frac{c_1}{c_1 + c_2} + \frac{c_3}{v_{\max}} \right) \quad (3)$$

where $\alpha \in [0, 1]$ is the interpolation parameter used in Eq 1. The first term $\frac{c_1}{c_1 + c_2}$ combines the distances to indicate which spline is closest, while $\frac{c_3}{v_{\max}}$ normalises the speed; both are weighted equally for spline selection. Initially, all variables are set near the median trajectory (easy task). As the agent’s

TABLE I
AVERAGE PERFORMANCE METRICS ON BARCELONA (4 SESSIONS PER MODEL). “—” = NO LAP COMPLETED (BIKE UNABLE TO COMPLETE THE TRACK).

Metric	Iter. 1150		Iter. 1900	
	SPDL	SAC	SPDL	SAC
Avg. lap time	1:32.85	—	1:30.75	1:31.32
Dist. to ideal (m)	2.99	4.59	3.15	2.89
Off-track time (s)	1.74	9.22	1.17	2.15
Falls	0	3	0	0
Collisions	0	0	0	0

TABLE II
TRACK GENERALIZATION WITH SPDL AFTER 1150 ITERATIONS.

Track	Avg. Lap	Dist. to ideal (m)	Off-track (s)	Avg. Lap (SAC)
Barcelona	1:32.85	2.99	1.74	-
Cremona	1:28.22	3.73	1.31	1:29:14
Phillip Island	1:27.49	3.10	4.35	-

performance exceeds a threshold δ , SPDL automatically shifts the distribution toward the target context (ideal line at full speed). The KL constraint $D_{\text{KL}}(p(c|\nu) \| p(c|\nu')) \leq \epsilon$ prevents abrupt transitions. We use SPDL version 2 [6], which requires setting only a single performance threshold δ , making it significantly easier to tune than the two-parameter version [5].

VI. EXPERIMENTAL EVALUATION

A. Setup

All experiments use SAC from Stable Baselines 3 (learning rate 2×10^{-5} , replay buffer 3×10^6 , batch size 512), with 10 parallel environments. Two algorithms are compared: **SAC** (baseline, trained on the ideal trajectory throughout) and **SPDL** (SAC + Self-Paced Curriculum). Training runs for up to 1900 iterations of 5000 steps each ($\approx 9.5\text{M}$ environment steps per run). The primary track is **Barcelona**; additional evaluation uses **Cremona** and **Phillip Island**. All experiments use the Ducati Panigale V4 R unless stated otherwise, in Hot Lap (time-trial) mode.

B. Deployment Results

Table I shows the key deployment metrics for the Barcelona circuit. At iteration 1150, the SAC-only model fails to complete a single lap, repeatedly losing control between the first and second track sectors. SPDL completes all laps with zero falls and an average time of 1:32.85. By iteration 1900, SAC has trained sufficiently to complete laps, with a best of 1:31.32; SPDL achieves 1:30.75 — a gain of 0.57 s. Over three consecutive laps, SPDL gains 0.77 s, accumulating advantage primarily in the initial part of the circuit.

Track generalization. Table II shows SPDL-trained agents on three different tracks at iteration 1150. In all cases, the agent completes laps with zero falls and zero wall collisions, demonstrating that the proposed architecture and curriculum scheme generalize across different track geometries. The higher off-track time on Phillip Island is due to brief excursions when the agent navigates a specific chicane at high speed.

TABLE III
CROSS-BIKE TRANSFER: BEST SINGLE LAP TIMES (MM:SS.SSS).

Motorbike	Barcelona	Cremona	Phil. Island
Ducati Panigale V4 R (trained)	1:30.24	1:27.80	1:26.01
Kawasaki Ninja ZX-10RR	1:31.10	1:27.98	1:28.75
Honda CBR1000RR	1:32.34	1:28.81	1:24.62
Yamaha R1	1:32.53	1:29.52	1:24.29
BMW M1000 RR	1:33.13	1:30.32	—

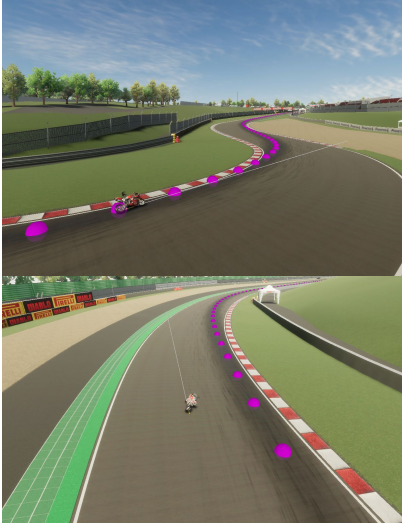


Fig. 4. Two trajectories followed by the motorbike. In the top one, the bike is able to follow the trajectory in a proper way. In the bottom one, the bike enters too fast into the corner and struggles to follow the ideal trajectory.

Motorbike generalization. We transferred the Ducati Panigale V4 R-trained model after 1900 iterations to all four remaining motorbike models without retraining. Table III shows the best lap times. The Kawasaki Ninja ZX-10RR transfers best, trailing the Ducati by only 0.87 s in Barcelona. Honda and Yamaha models show competitive performance, and even outperform the Ducati on Phillip Island due to better handling of a track-specific corner. Only the more aggressive and hard-to-drive BMW M1000 RR struggles on Phillip Island, failing to complete the lap. These results confirm that the learned policy captures generalizable motorbike control priors, not just bike-specific dynamics.

C. Qualitative results

While early results are promising, a notable limitation of the current system is that the agent prioritizes speed over trajectory precision, overshooting corners. An example of this is in Fig. 4. Against this, future work should incorporate explicit velocity planning signals or trajectory-adherence rewards during high-curvature segments.

Additionally, the current framework trains a separate agent per track; extending to multi-track generalization — where the agent learns geometry-agnostic driving priors — is an important direction. Finally, the asymmetric actor-critic architecture of [3], where the critic receives privileged global information

during training, could further improve sample efficiency, as could vision-based inputs for a more realistic sensing modality.

VII. CONCLUSION

We presented the preliminary results of a Self-Paced Curriculum Reinforcement Learning framework for autonomous superbike racing in VRider SBK. By integrating SAC with SPDL and a motorbike-tailored state-action-reward formulation, our agent learns to race a motorbike in time-trial scenarios, outperforming vanilla SAC in training efficiency and lap time. The framework generalizes across multiple tracks and motorbike models, establishing, to the best of our knowledge, the first RL baseline for autonomous racing bikes. Future works will improve the current framework, testing performance in competitive racing settings.

ACKNOWLEDGMENT

The authors want to thank the European Union’s Horizon 2020 RIA program ELISE GA No 951847.

REFERENCES

- [1] F. Fuchs, Y. Song, E. Kaufmann, D. Scaramuzza, and P. Dür, “Superhuman performance in gran turismo sport using deep reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4257–4264, 2021.
- [2] P. R. Wurman, S. Barrett, K. Kawamoto, J. MacGlashan, K. Subramanian, T. J. Walsh, R. Capobianco, A. Devlic, F. Eckert, and F. Fuchs, “Outracing champion gran turismo drivers with deep reinforcement learning,” *Nature*, vol. 602, no. 7896, pp. 223–228, 2022.
- [3] M. Vasco, T. Seno, K. Kawamoto, K. Subramanian, P. R. Wurman, and P. Stone, “A super-human vision-based reinforcement learning agent for autonomous racing in gran turismo,” in *Proceedings of the 2024 Reinforcement Learning Conference (RLC)*, 2024.
- [4] H. Lee, T. Seno, J. J. Tai, K. Subramanian, K. Kawamoto, P. Stone, and P. R. Wurman, “A champion-level vision-based reinforcement learning agent for competitive racing in gran turismo 7,” *IEEE Robotics and Automation Letters (RA-L)*, 2025.
- [5] P. Klink, C. D’Eramo, J. R. Peters, and J. Pajarinen, “Self-paced deep reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9216–9227, 2020.
- [6] P. Klink, H. Abdulsamad, B. Belousov, C. D’Eramo, J. Peters, and J. Pajarinen, “A probabilistic interpretation of self-paced learning with applications to reinforcement learning,” *Journal of Machine Learning Research*, vol. 22, no. 182, pp. 1–52, 2021.
- [7] P. Klink, H. Yang, C. D’Eramo, J. Peters, and J. Pajarinen, “Curriculum reinforcement learning via constrained optimal transport,” in *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 2022, pp. 11 341–11 358.
- [8] Y. Song, H. Lin, E. Kaufmann, P. Dür, and D. Scaramuzza, “Autonomous overtaking in gran turismo sport using curriculum reinforcement learning,” in *Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 9403–9409.
- [9] B. Grooten, P. MacAlpine, K. Subramanian, P. Stone, and P. R. Wurman, “Out-of-distribution generalization with a sparc: Racing 100 unseen vehicles with a single policy,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026.
- [10] A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar, and D. Lange, “Unity: A general platform for intelligent agents,” *arXiv preprint arXiv:1809.02627*, 2020. [Online]. Available: <https://arxiv.org/pdf/1809.02627.pdf>
- [11] G. Brockman, V. Cheung, L. Petteersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “Openai gym,” *arXiv preprint arXiv:1606.01540*, 2016.
- [12] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.